Indirection

Indirection Principle

Add a middleman to decouple

DEFINITION

Assign responsibility to an intermediate object so two components stay decoupled from each other.

Indirection introduces a middle object so two components never refer to each other directly. Instead of A knowing B, both know an intermediary — an adapter, mediator, controller, or facade — that carries the relationship. The mediator absorbs the coupling, leaving each side dependent only on the seam in the middle.

WHY IT MATTERS

Direct links between volatile or numerous components create a tangled web where every change ripples outward. A middleman gives you one place to reroute, intercept, adapt, log, or swap a relationship without touching either end. The cost is a level of redirection — one more hop to read and debug — so it pays only where coupling genuinely hurts.

Vasyl Krupka · Ukraine

— How it works

Intermediary	The mediating object that carries the relationship so the two parties never bind directly.
Parties	The components being kept apart — each depends on the intermediary, not on the other.
Effect	Lower direct coupling and a single seam to swap, adapt, or intercept the interaction.

HOW TO APPLY

1. Spot two components that depend on each other but shouldn't.
2. Introduce a middle object to own that relationship (mediator, adapter, facade).
3. Point both sides at the intermediary instead of at each other.
4. Add indirection only where coupling is real — not "just in case".

LITMUS TEST

Do two components reference each other directly when they should not? Would a seam let you swap, adapt, or intercept the link? If so, insert an intermediary.

— Smell → fix

Widgets call each other directly — a web of references that grows with every new widget:

```
class Button {
onClick() {
this.list.refresh(); this.label.update() // knows peers directly
}
}
```

↓ INSERT A MIDDLEMAN

```
class Mediator {
notify(sender: object, event: string) { /* route to peers */ }
}
class Button {
onClick() { this.mediator.notify(this, 'click') } // only Mediator
}
```

Components depend on the mediator, not on each other. Adding a widget touches one place — but every interaction now hops through the middle, so use it where coupling truly hurts.

USE WHEN

- ✓ Two components shouldn't know each other
- ✓ Insert a seam — adapter, mediator, controller

AVOID WHEN

- ▲ Layers of indirection "just in case" — hurts clarity

— Trade-offs & pitfalls

PROS

+ Low coupling
+ Swappable, interceptable

CONS

− More moving parts
− Harder to trace

COMMON PITFALLS

▲ Speculative layers of indirection that add hops without removing any real coupling.
▲ A mediator that grows into a god object as it absorbs everyone's logic.
▲ Indirection that hurts traceability — stack traces and "go to definition" stop being useful.

WHERE YOU SEE IT

- A Mediator coordinating UI widgets so they don't reference each other.
- Adapter / Facade wrapping a subsystem; an API gateway in front of services.
- A Repository between domain and database; a message broker between producers and consumers.

SEE ALSO

Mediator	a GoF pattern that is Indirection applied to many-to-many object talk
Facade	indirection that hides a whole subsystem behind one entry point
Protected Variations	Indirection is a common way to build a stable, protective seam
Low Coupling	the property Indirection buys — at the cost of more moving parts

— Interview & sources

Indirection vs Protected Variations?

Indirection adds a middleman to decouple two parties; Protected Variations wraps a variation point behind a stable interface. Indirection is a common means to achieve PV.

When does indirection hurt?

When added "just in case": each layer is one more hop to read and debug. Indirection trades traceability for flexibility — pay only where coupling is real.

Examples in real frameworks?

DI containers, message buses, adapters, API gateways, repositories — all insert an intermediary so callers and providers stay decoupled.

Is more indirection always better design?

No — each layer trades traceability for flexibility. Indirection is valuable exactly where two parts should evolve independently; everywhere else it's overhead that makes the code harder to follow.

REMEMBER

Put a middleman between two things that should not know each other.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Indirection

GoF — "Design Patterns" (1994): Mediator, Facade, Adapter